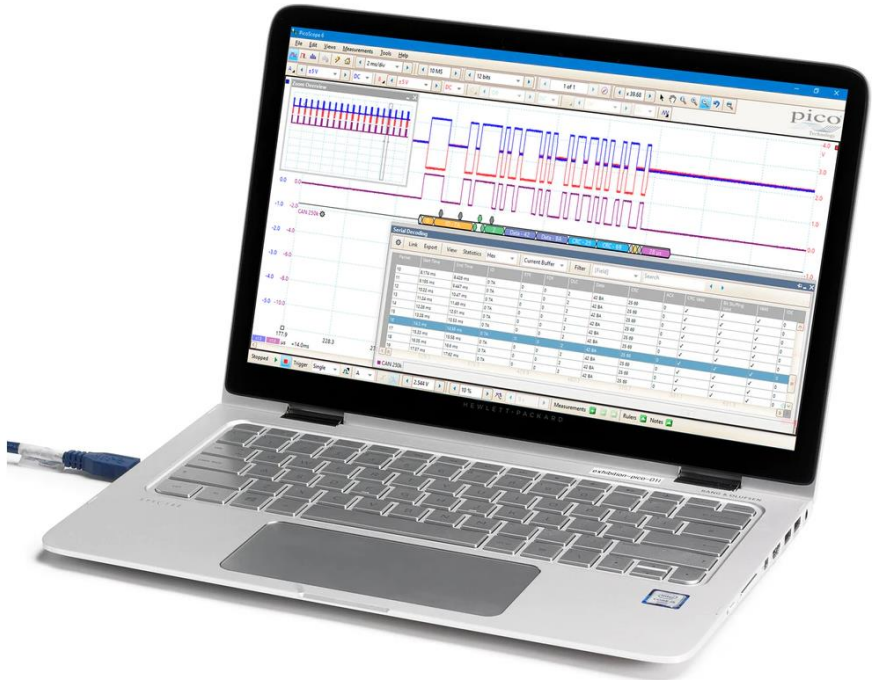
Telemetry and Telecommand(UART)

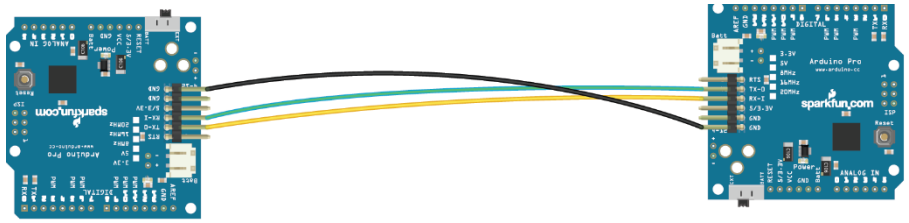
Telemetry &
Telecommand

UART

(HAL_UART)



Protocols



Wire based Protocl.

- In computer networking, data can be exchanged through some set of rules where the transmission of data is done through wire like normal wires, coaxial cable , fiber optics.
- A wired setup uses physical cables to transfer data between different devices and computer systems.
- There are numerous data exchange prtocols based on physical wires, like USB, Serial Communication, I2c, CAN,..etc.

- Serial Bus
- I2c Bus
- CAN Bus
- SPI Bus

..

..

..

UART

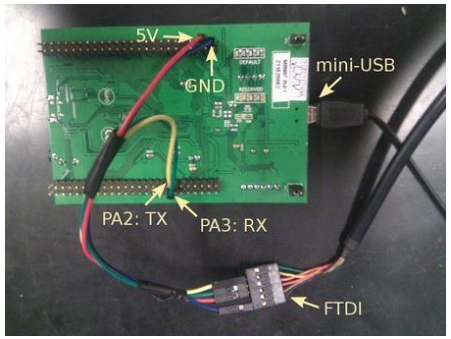
Universal Asyncrhonous Reciever Transmitter

- An RS-232 **serial port** was once a standard feature of a personal computer, used for connections to modems, printers,mice, data storage, uninterruptible power supplies, and other peripheral devices.

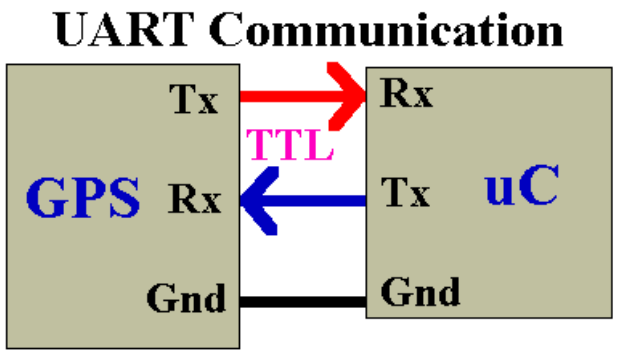
- The universal asynchronous receiver/transmitter (UART) takes bytes of data and transmits the individual bits in a sequential fashion.[1] At the destination, a second UART re-assembles the bits into complete bytes.

UART

- Universal Asyncrhonous Reciever Transmitter



- Asyncrhronous is a keyword here means that there is no dedicated clock signal between Transmitter and Reciever.
- Both Reciever and Transmitter need to know the bit rate or the speed before the communication is started.
- Thats the most important consideration when you are doing the UART based communication.



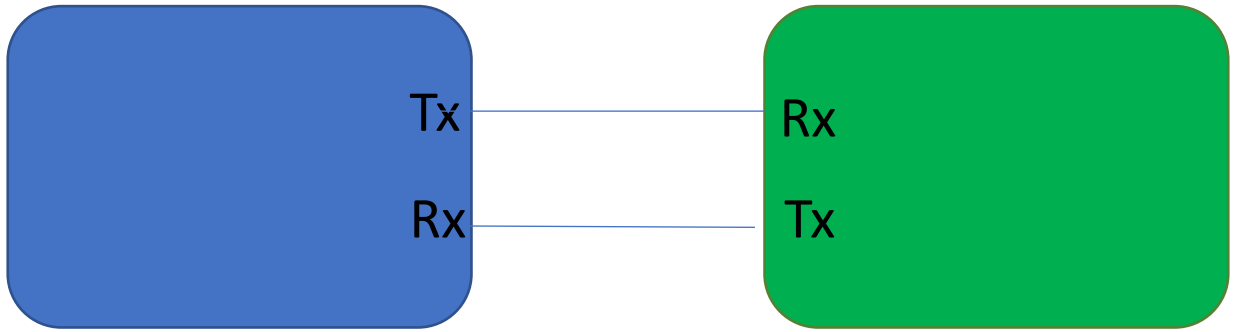
UART

- Universal Asyncrhonous Reciever Transmitter

As the *R* and *T* in the acronym dictate, UARTs are responsible for both sending and receiving serial data.

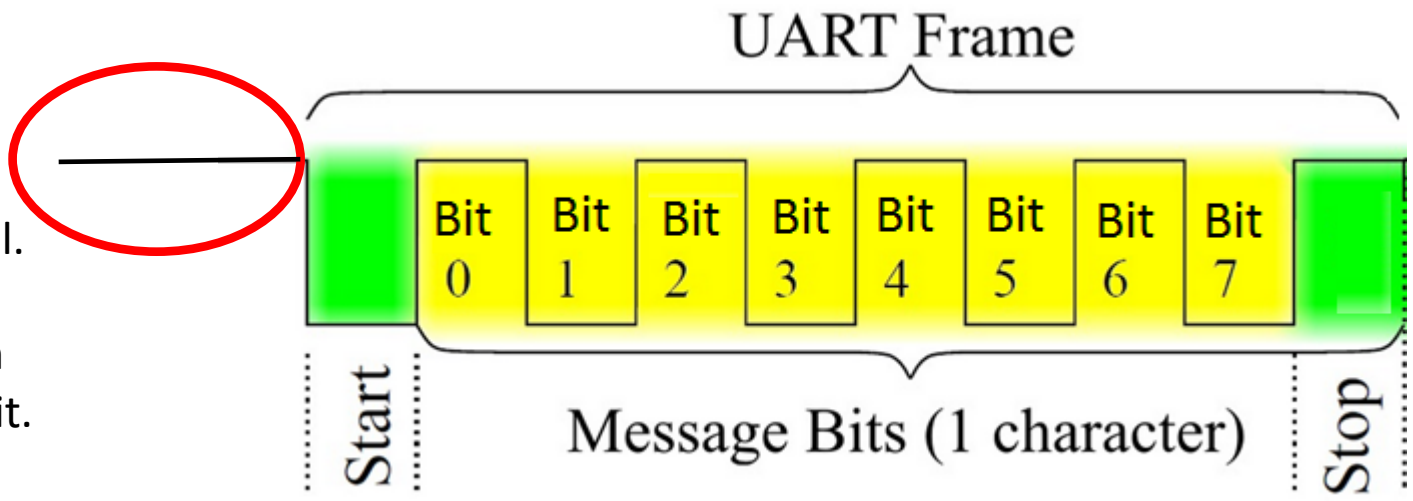
On the transmit side, a UART must create the **data packet** - and send that packet out the TX line with precise timing (according to the set baud rate).

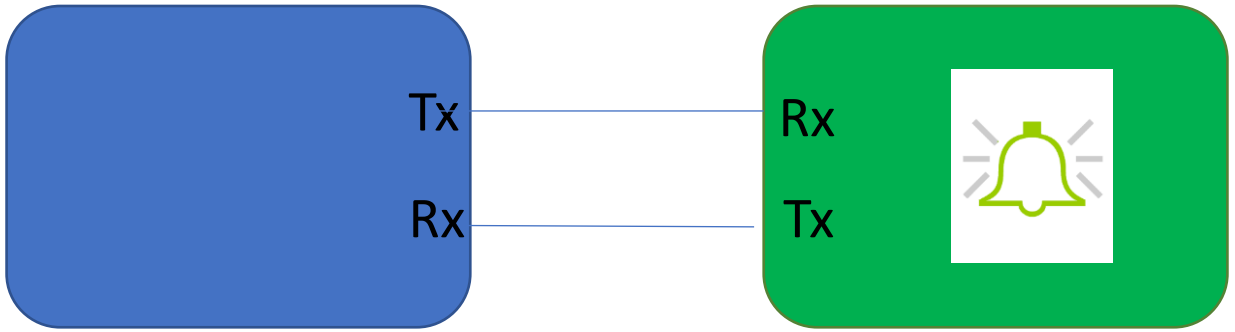
On the receiver end, the UART has to sample the RX line at rates according to the expected baud rate, and pick the data.



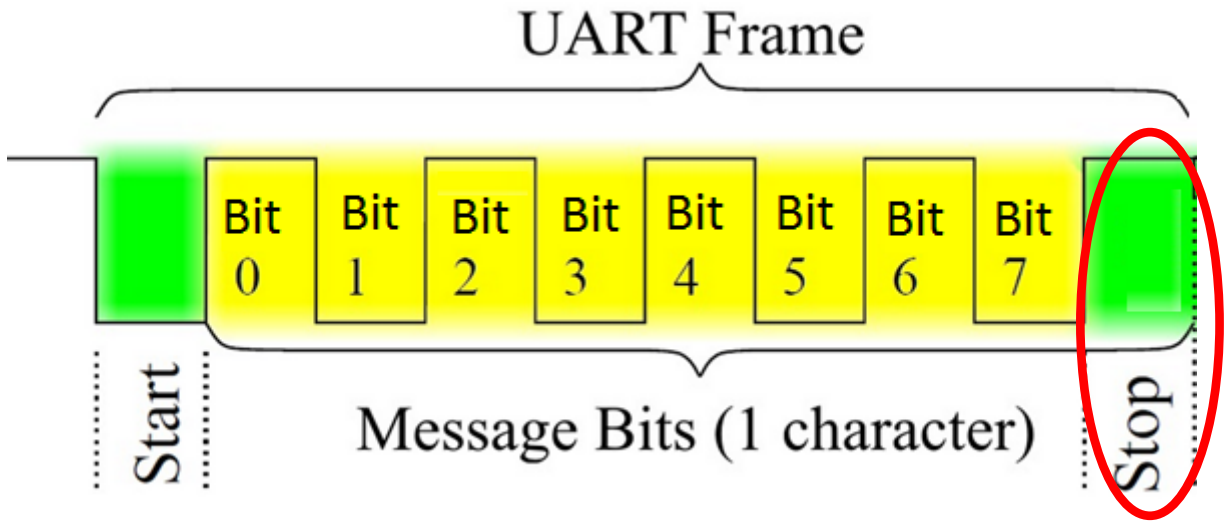
UART

- Transmit waveform at the idle State is high level.
- When you are ready to transmit the signal then drop the Tx line low that Designates the start bit.
- The Start Bit is used to **alert the receiver** that a word of data is about to be sent, and to force the clock in the receiver into synchronization with the clock in the transmitter.

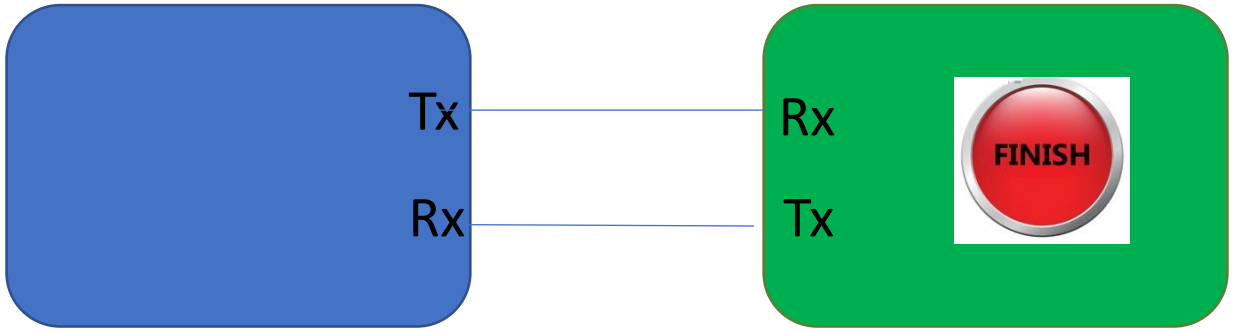




UART

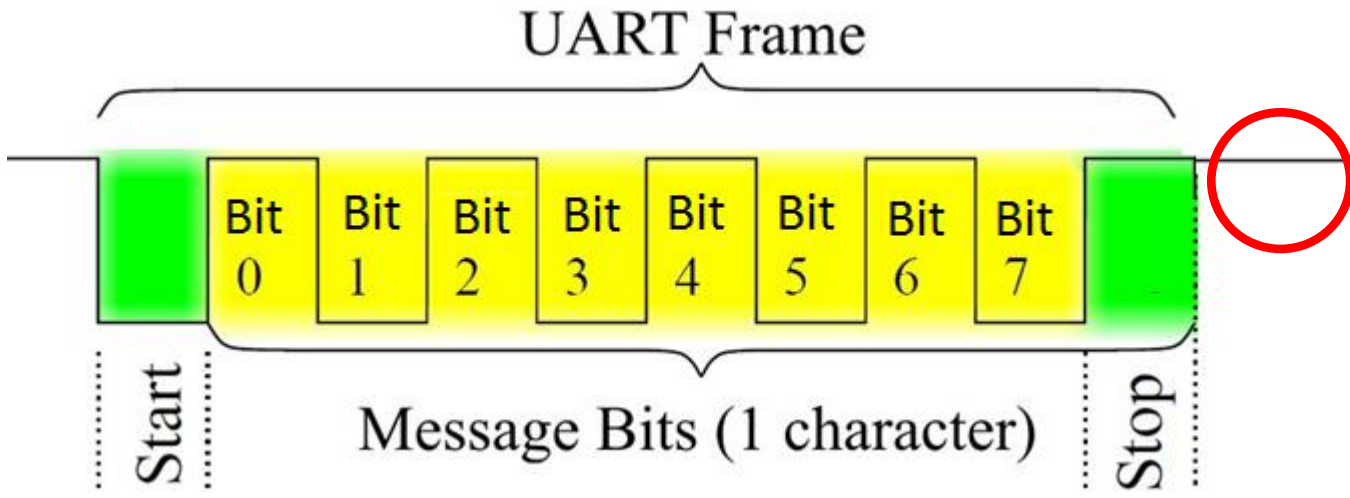


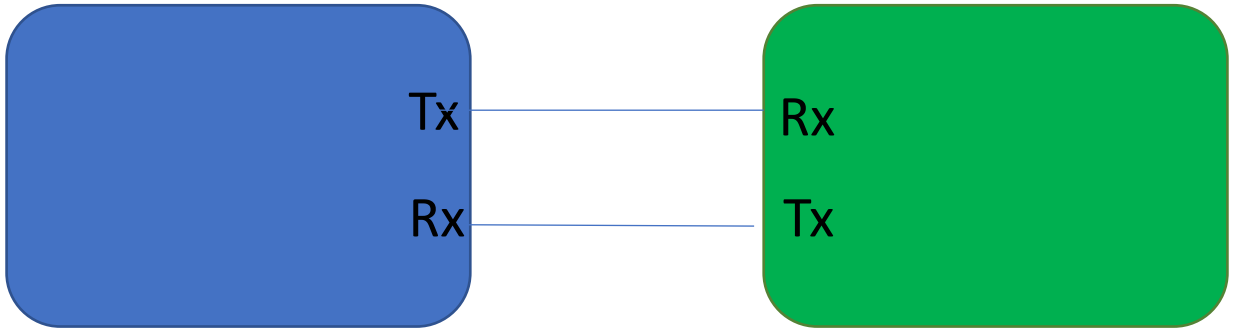
- Then we transmit a single bit which is designated as STOP bit. As it is shown here stop bit is the opposite state of the START bit.
- Atleast one Stop Bit is sent by the transmitter.



UART

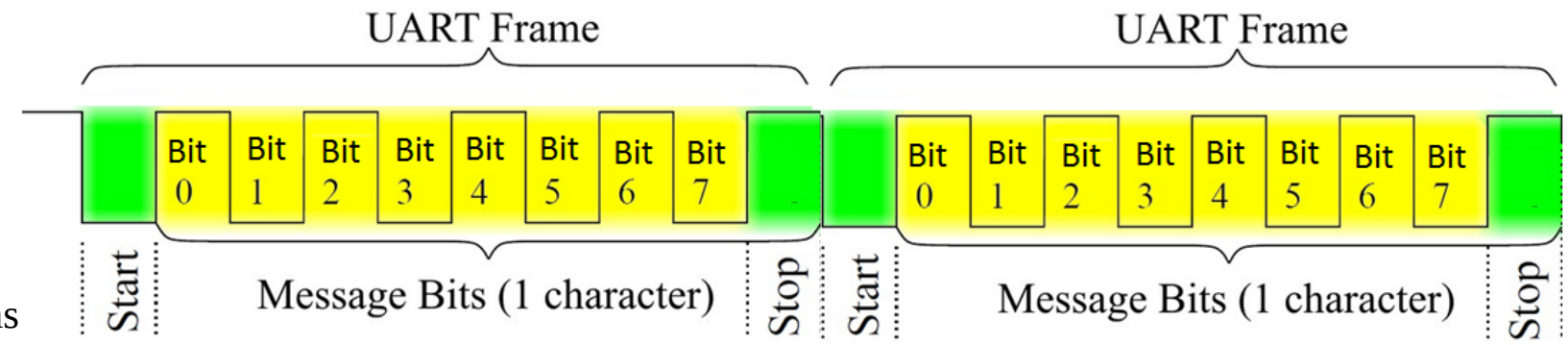
- After we transmit the STOP bit we are back to the Idel State.
- Because asynchronous data is “self synchronizing”, if there is no data to transmit, the transmission line can be idle.





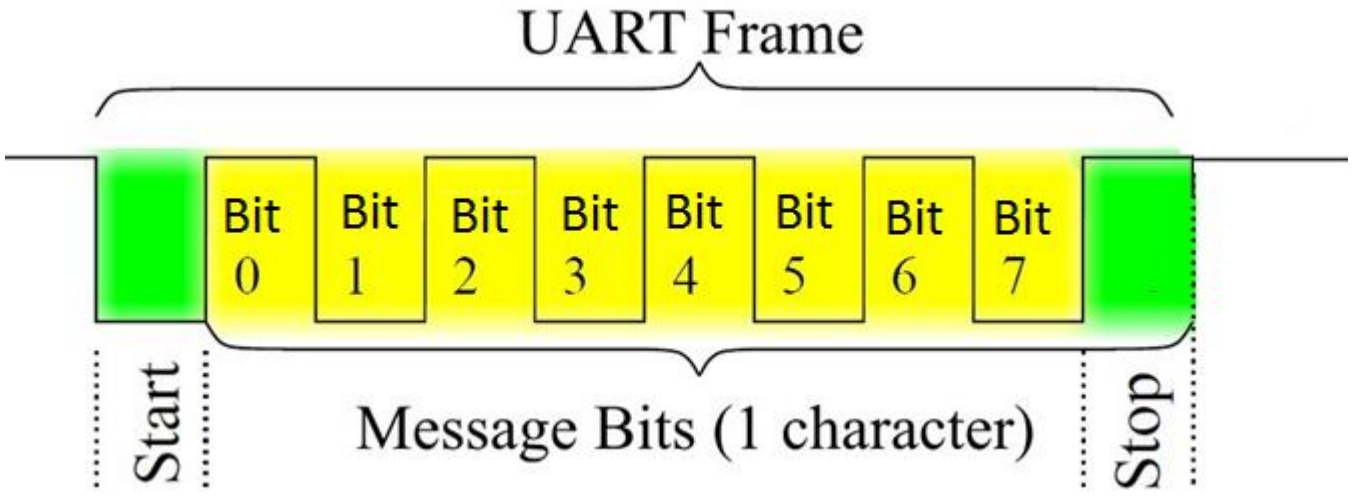
UART

- If another word is ready for transmission, the Start Bit for the new word can be sent as soon as the Stop Bit for the previous word has been sent.



UART

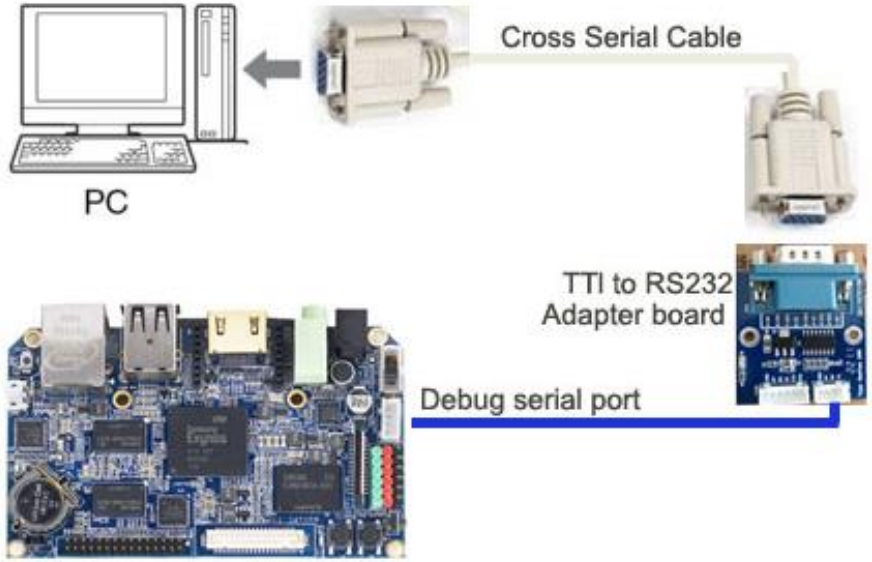
- So in total we have 10 bits. 8 data bits and 2 frame bits.
- Baud Rate means bit transmitted Per second.
- So for 115200 baud we are transmitting 115.2 kilo bits per second.



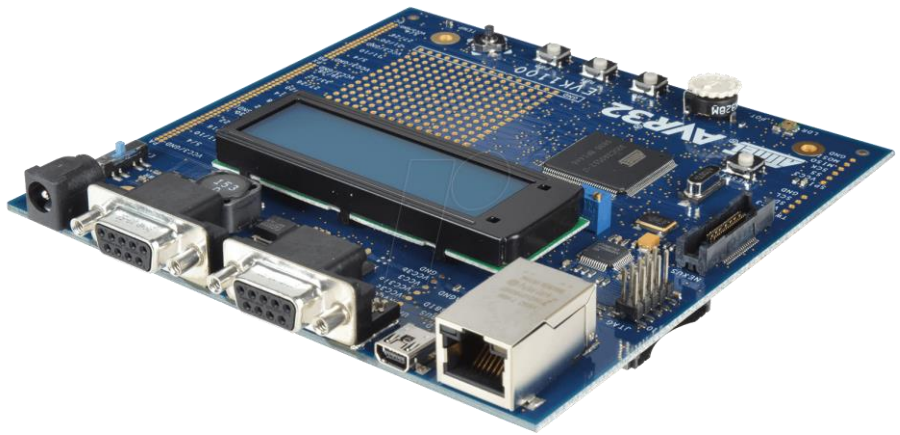
UART

Used in Debugging the Microcontroller Application

- Newer microcontrollers have a built in UART giving you a virtually free debug tool that uses minimal resources and needs very little software coding.
- Even through it takes time to output a character it is a useful debug tool as you can output the value of a variable to see what the microcontroller is really doing.



UART

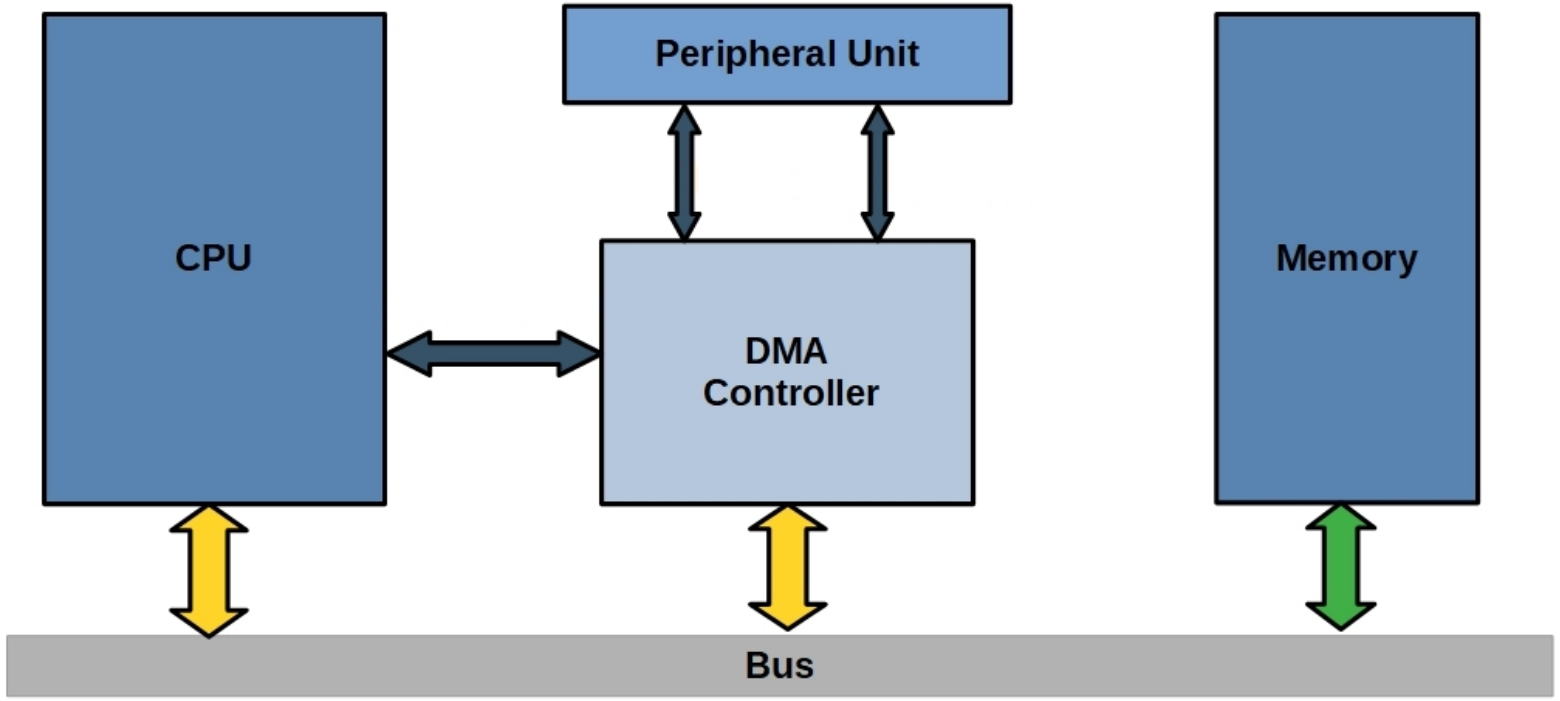


Disadvantages.

- Speed
- No Network based on more than two devices.
- Limited Number of UART ports.

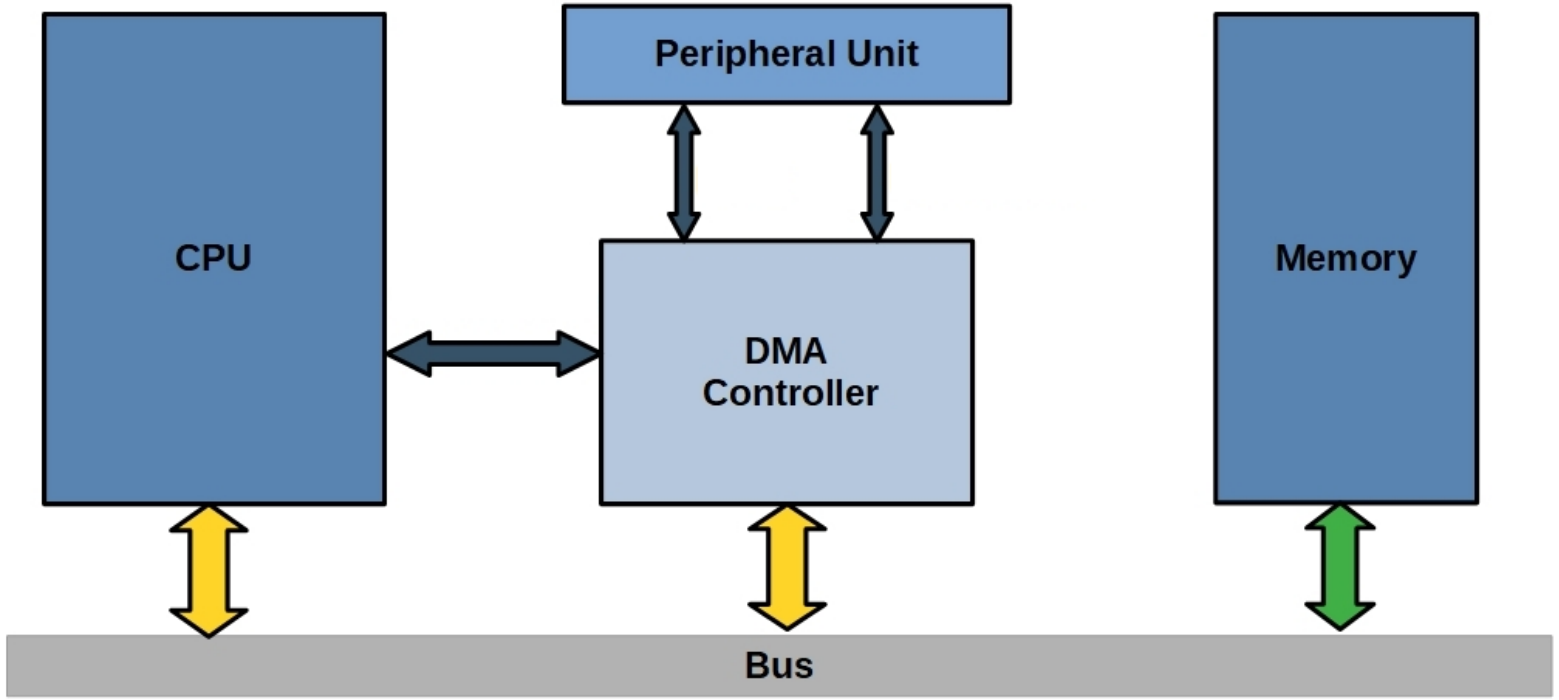
DMA in Microcontrollers

Direct memory access (**DMA**) is a feature of computer systems that allows certain hardware subsystems to access main system memory (random-access memory) independent of the central processing unit (CPU).



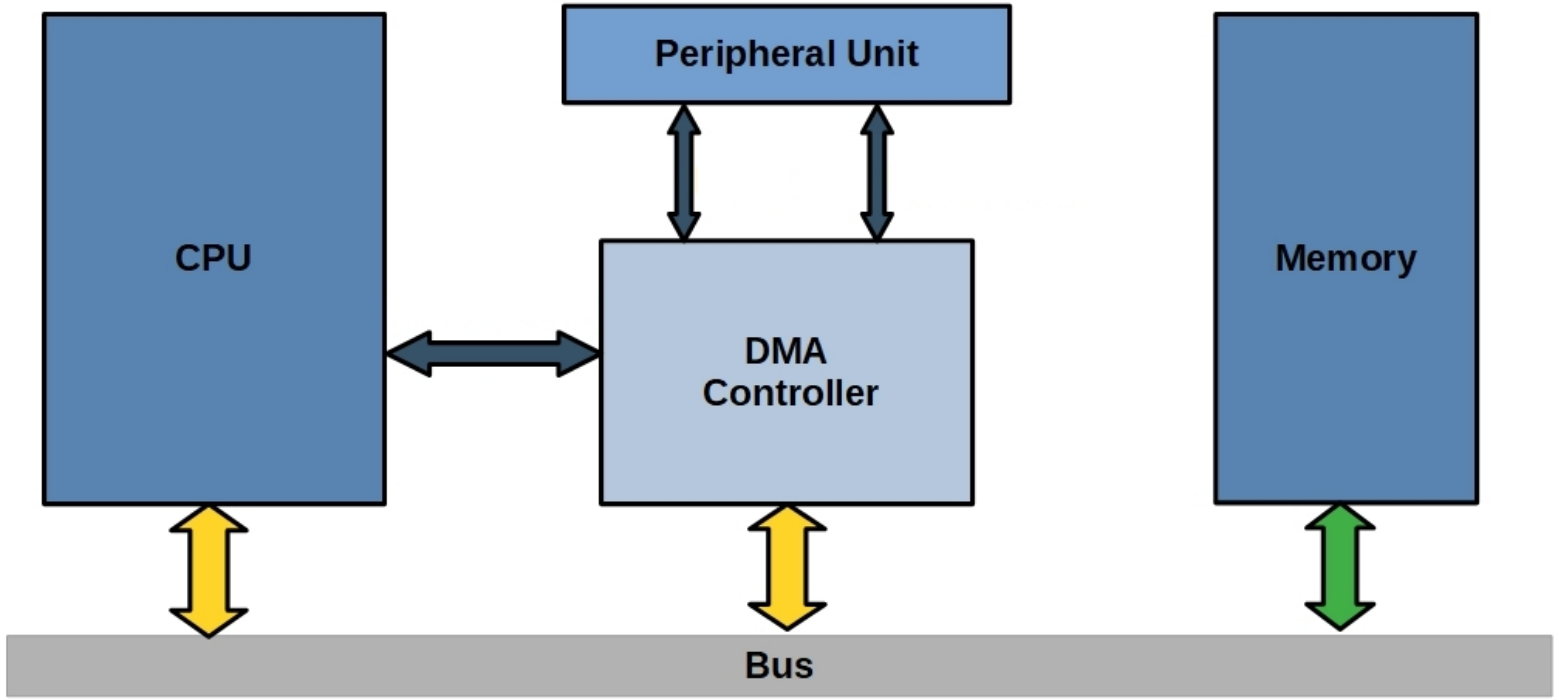
DMA in Microcontrollers

- Without DMA, when the CPU is using programmed input/output, it is typically fully occupied for the entire duration of the read or write operation, and is thus unavailable to perform other work.

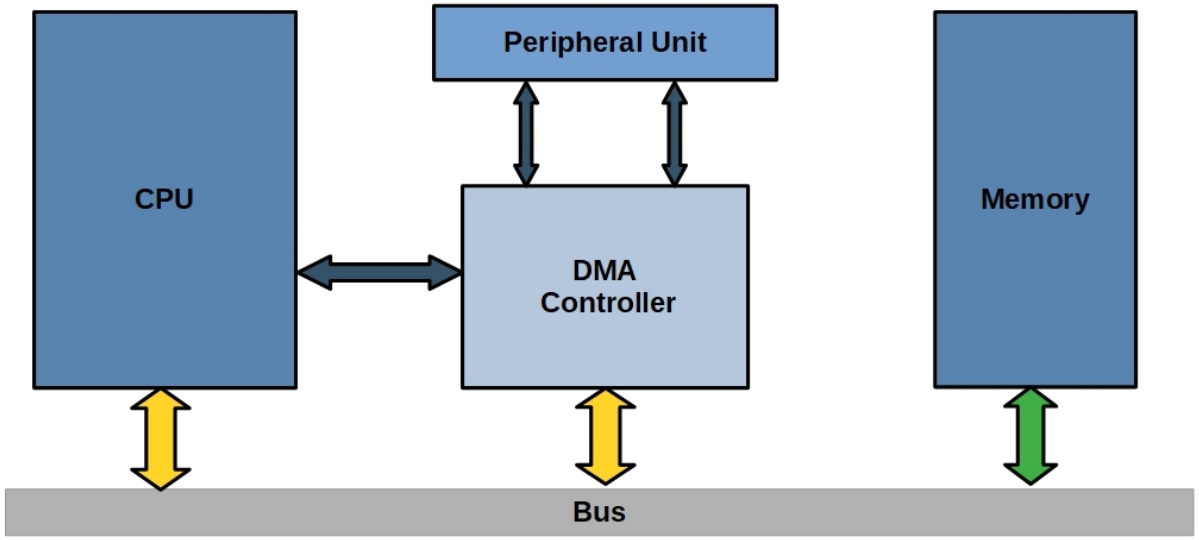


DMA in Microcontrollers

- With DMA, the CPU first initiates the transfer, then it does other operations while the transfer is in progress, and it finally receives an interrupt from the DMA controller (DMAC) when the operation is done.



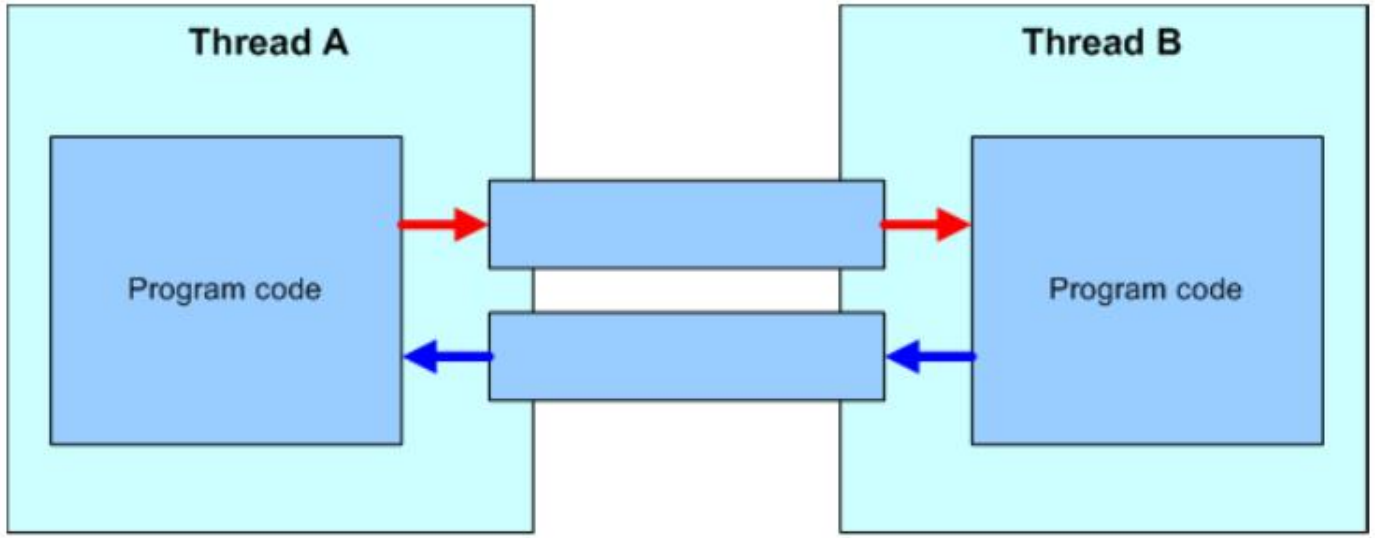
DMA in Microcontrollers



- This feature is useful at any time that the CPU cannot keep up with the rate of data transfer, or when the CPU needs to perform work while waiting for a relatively slow I/O data transfer.

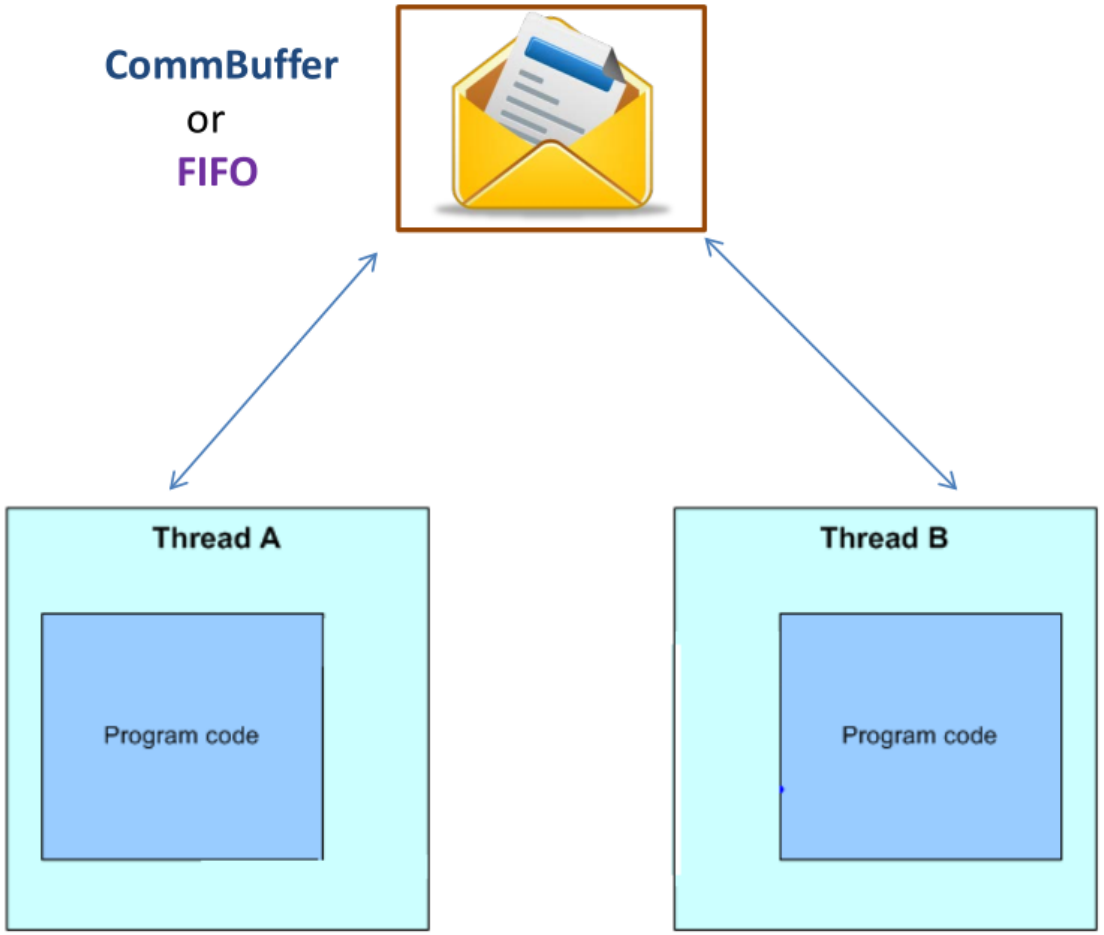
- disk drive controllers, graphics cards, network cards and sound cards ..etc all uses DMA

Thread Communication



The communication between two threads can bee realized via a CommBuffer or a FiFo (First in first out). For that a CommBuffer or a Fifo has to be defined outside a thread so that both threads can access it.

Thread Communication



Thread Communication

CommBuffer

A CommBuffer is a double buffer with **only one writer** and **only one reader**. Both can work concurrently. The writer may write at any time. The reader gets the newest consistent data (eg. the last complete written record). The type of the CommBuffer can be defined. Not using a CommBuffer is risky, because maybe the data is half written in the shared variable while the thread is interrupted. In this case the receiver thread gets inconsistent data.

Sender

```
CommBuffer<int> buf;

class Sender : public Thread {
...
  PRINTF("Writing %d\n", cnt);
  buf.put(cnt);
...
}
```

CommBuffer of type Integer

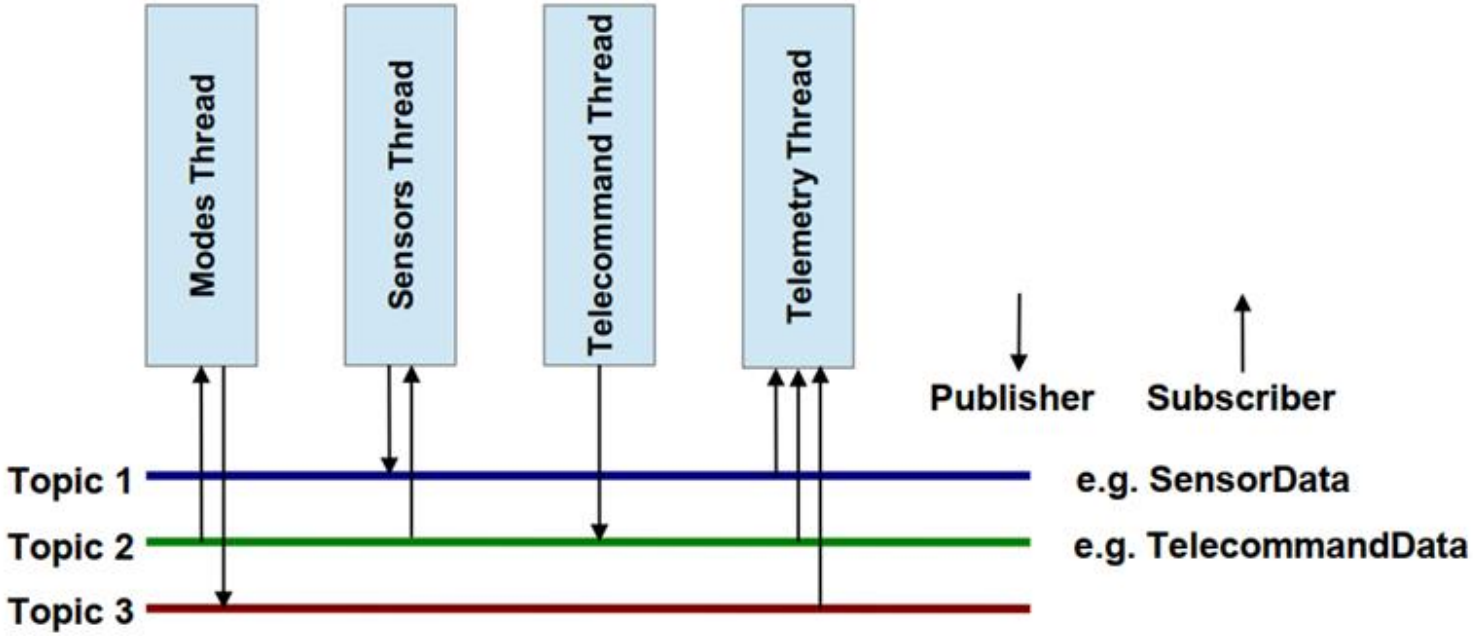
thread puts local counter data into the CommBuffer

Reciever

```
class Receiver : public Thread {
...
  buf.get(cnt);
  PRINTF("Reading %d\n", cnt);
...
}
```

thread gets counter data from the CommBuffer and saves it into local variable

RODOS (Realtime Onboard Dependable Operating System):




```
struct SensorData{
float gx, gy, gz; // x, y, and z axis readings of the gyroscope in deg/s
float temperature; // on-board temperature reading in degree
};
Topic<SensorData> SensorDataTopic(-1, "Sensor Data");
```

```
//Publisher Side
SensorData sensorDataPublisher;
SensorDataTopic.publish(sensorDataPublisher); //publish the message to all listeners.
```

If anybody wants to publishes (boradcasts) the SensorData then it should make an object of the concerned topic and then use the publish Method of the topic class

```
//Subscriber Side
CommBuffer<SensorData> SensorDataBuffer;
Subscriber SensorDataSubscriber(SensorDataTopic, SensorDataBuffer);
```

These 2 lines will connect Commbuffer with topic, If anybody publishes (boradcasts) the SensorData then the buffer will automatically get it

```
SensorData sensorDataReceiver;
SensorDataBuffer.get(sensorDataReceiver); //Get the next consistent message from the CommBuffer.
```

If anybody wants to recieve the SensorData then it should make an object of the concerned topic and then use get Method of the buffer class

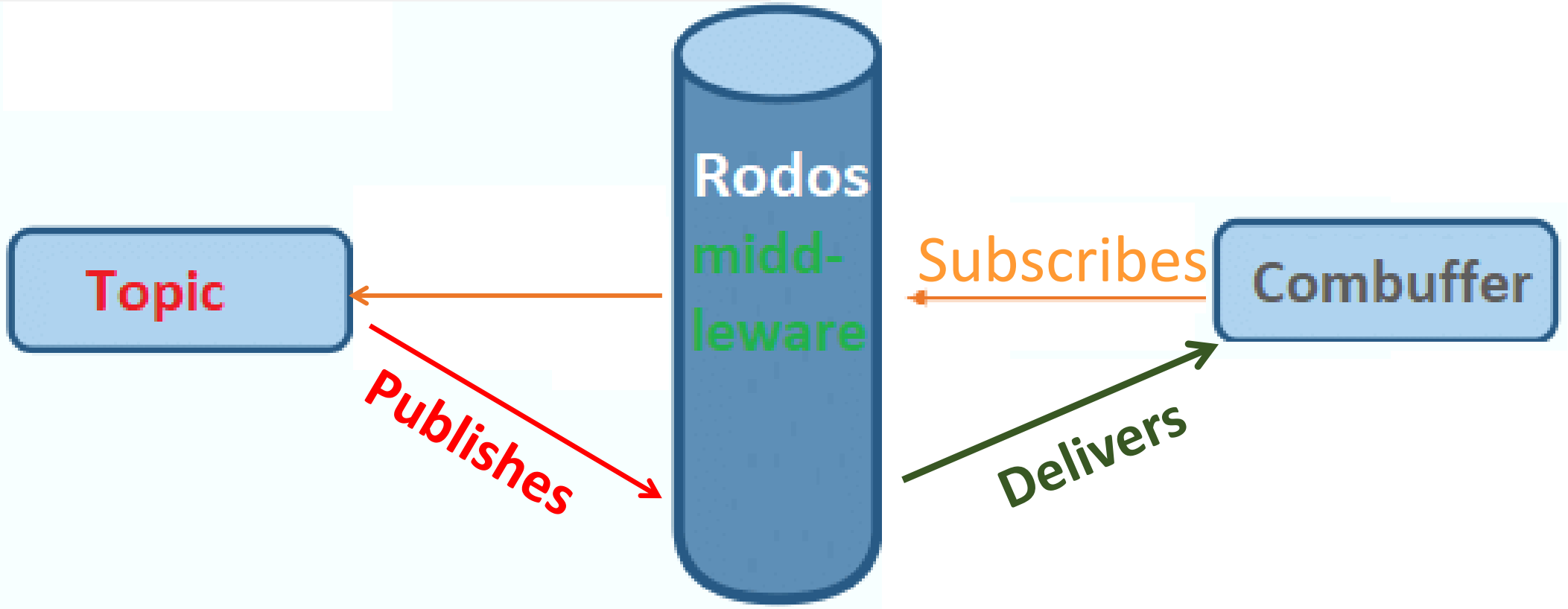
Middleware in RODOS

```
struct sSensorData{

        float gx, gy, gz;
        float pitch, roll, yaw;

};
```

```
Topic<sSensorData> SensorDataTopic(-1, "Sensor Data");
```

```
CommBuffer<sSensorData> SensorDataBuffer;
```

```
Subscriber SensorDataSubscriber(SensorDataTopic, SensorDataBuffer);
```



sprintf function in C

```c
int sprintf(char *str, const char *format, ...)
```

```c
#include <stdio.h>
#include <math.h>

int main () {
   char str[80];

   sprintf(str, "Value of Pi = %f", M_PI);
   puts(str);

   return(0);
}
```

Let us compile and run the above program, this will produce the following result −

```
Value of Pi = 3.141593
```

sprintf function in C

```c
// Example program to demonstrate sprintf()
#include <stdio.h>
int main()
{
    char buffer[50];
    int a = 10, b = 20, c;
    c = a + b;
    sprintf(buffer, "Sum of %d and %d is %d", a, b, c);

    // The string "sum of 10 and 20 is 30" is stored
    // into buffer instead of printing on stdout
    printf("%s", buffer);

    return 0;
}
```

Output

```
Sum of 10 and 20 is 30
```

sscanf function in C

```c
/* sscanf example */
#include <stdio.h>

int main ()
{
  char sentence []="Rudolph is 12 years old";
  char str [20];
  int i;

  sscanf (sentence,"%s %*s %d",str,&i);
  printf ("%s -> %d\n",str,i);

  return 0;
}
```

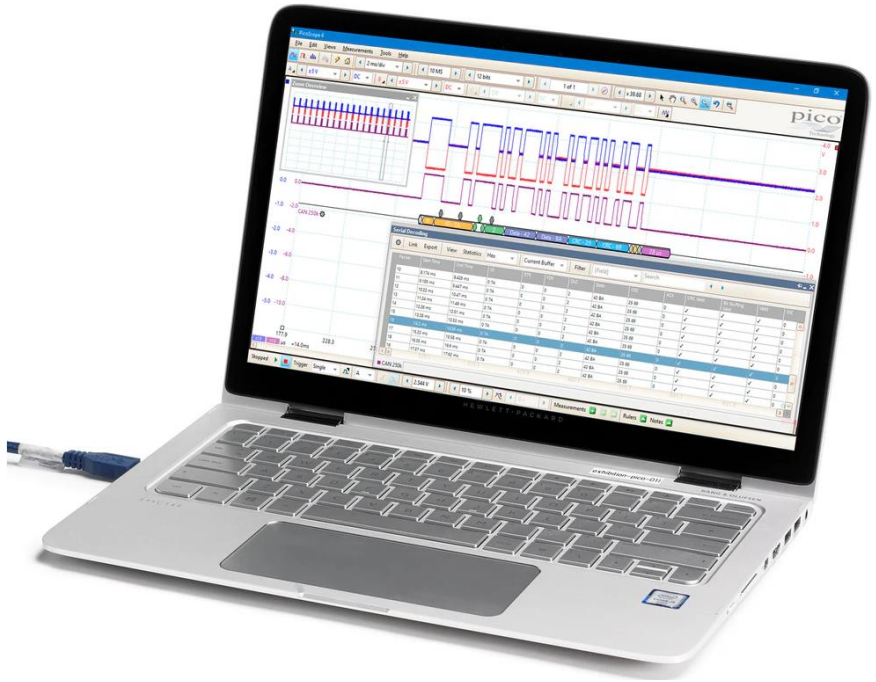
Output:

```
Rudolph -> 12
```

Telemetry &
Telecommand

UART

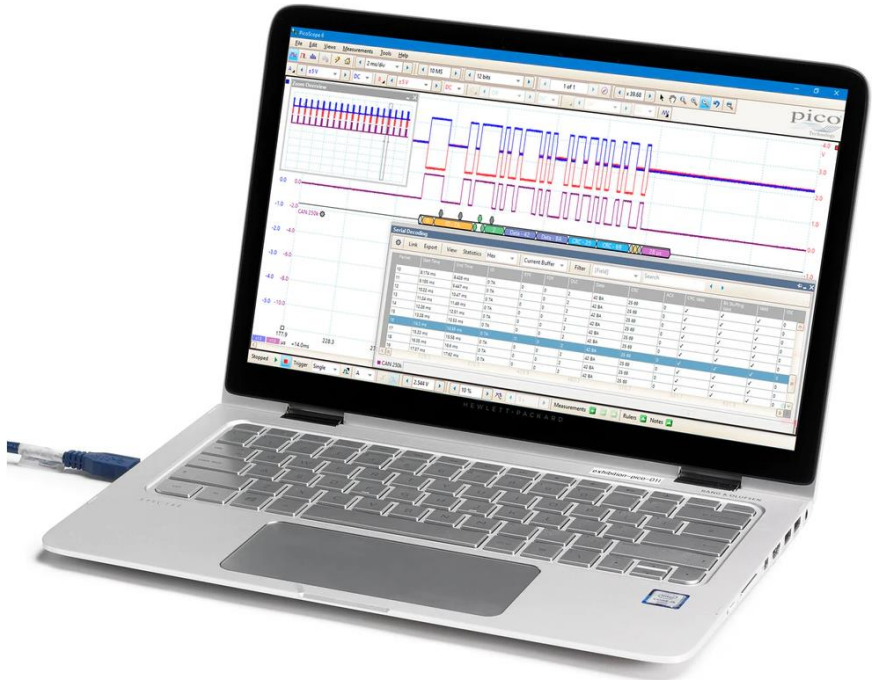
(HAL_UART)



Telemetry &
Telecommand

UART

(HAL_UART)



RODOS HAL (UART):

```
class HW_HAL_UART; /* hal_uart.h */
class HAL_UART : public GenericIOInterface {
private:
HW_HAL_UART* context;

/* public functions */
public:
HAL_UART(UART_IDX uartIdx);

/* Initialization of uart interface: mode 8N1, no HW flow control, USART_IT_RXNE & USART_IT_TXE Interrupts enable */
int init(unsigned int baudrate = 115200);

/* disable interface and set all its registers and pins to its reset state */
void reset();

 /* Configuration of uart interface AFTER initialization */
int config(UART_PARAMETER_TYPE type, int paramVal);

int status(UART_STATUS_TYPE type);

bool isWriteFinished();
bool isDataReady();

/* write()
 * non-blocking -> returns only if "sendBuf" is sent */
int write(const char* sendBuf, int len);

/* read()
 * non-blocking -> returns when uart-receive-buffer is empty or "recBuf" is full */
int read(char* recBuf, int maxLen);

int getcharNoWait(); // returns character on success else -1
int putcharNoWait(char c); // returns c on success else -1
};
```

paramVal for DMA enabling is represent the number of bytes expected to be received before a DMA Interrupt can occur. Otherwise a timeout of 1ms will take place.

```
enum UART_PARAMETER_TYPE {
UART_PARAMETER_BAUDRATE,
UART_PARAMETER_HW_FLOW_CONTROL,
UART_PARAMETER_ENABLE_DMA};
```

```
enum UART_STATUS_TYPE {
UART_STATUS_RX_BUF_LEVEL,
UART_STATUS_RX_ERROR
};
```

Example:

```
HAL_UART BT2UART(UART_IDX2); /* declare HAL_UART for UART_IDX2 = UART2 (BlueTooth Module), please refer to hal_uart.h for correct UART mapping */
BT2UART.init(921600);        /* initialization of the HAL object*/
BT2UART.write("Hello Rodos", 11);  /* write "Hello Rodos" in uart3 */
```

Exercise